



A First Taste of Java

Type	Lecture
Date	@January 3, 2022
Lecture #	1
Lecture URL	https://youtu.be/ULpvhw2hmSg
Notion URL	https://21f1003586.notion.site/A-First-Taste-of-Java-c8303394228e4344969df9d13f8c4bd0
Week #	2

Getting Started

- In Python

```
print('hello, world')
```

- In C ...

```
#include <stdio.h>
main() {
    printf("hello, world\n");
}
```

- And then there is Java ...

```
public class helloworld {
    public static void main(String[] args) {
        System.out.println("hello, world");
    }
}
```

Why so complicated?

```
public class helloworld {
    public static void main(String[] args) {
        System.out.println("hello, world");
    }
}
```

- All code in Java lies within a class
 - No free floating functions, unlike Python and other languages
 - Modifier `public` specifies visibility
- How does the program start?
 - Fix a function name that will be called by default
 - From C, the convention is to call this function `main()`
- Need to specify input and output types for `main()`
 - The signature of `main()`
 - Input parameters is an array of strings
 - Command line arguments
 - No output, hence the `main` function has the return type `void`
- Visibility
 - Function has to be available to run from outside the class
 - Modifier `public`
- Availability
 - Functions defined inside classes are attached to the objects
 - How can we create an object before starting?
 - Modifier `static` — function that exists independent of dynamic creation of objects, *belongs to the class*
- The actual operation
 - `System` is a public class
 - `out` is a ***stream*** object defined in `System`
 - Like a file handle
 - Note that `out` must also be `static`
 - `println()` is a method associated with streams
 - Prints argument with a newline, like Python `print()`
 - Adds a newline character `\n` at the end of the statement
- Punctuation `{`, `}`, `;` to delimit the blocks, statements
 - Unlike layout and indentation in Python

Compiling and running Java code

- A Java program is a collection of classes
- Each class is defined in a separate file with the same name, with extension `.java`
 - Class `helloworld` in `helloworld.java`
- Java programs are usually interpreted on **Java Virtual Machine (JVM)**
 - JVM provides a uniform execution environment across OSes
 - Semantics of Java is defined in terms of JVM, OS-independent
 - It came with a slogan “Write once, run anywhere”
- `javac` compiles into JVM **bytecode**
 - `javac helloworld.java` creates bytecode file `helloworld.class`
- `java helloworld` interprets and runs bytecode in `helloworld.class`
- Note:
 - `javac` requires file extension `.java`
 - `java` should not be provided file extension `.class`

- `javac` automatically follows dependencies and compiles all the classes required
 - Sufficient to trigger compilation for class containing `main()`

Summary

- The syntax of Java is comparatively heavy
- Many modifiers — unavoidable overhead of object-oriented design
 - Visibility: `public` and `private`
 - Availability: all functions live inside objects, need to allow `static` definitions
 - Will see more modifiers as we go along
- Functions and variable types have to be declared in advance
- Java compiles into code for a virtual machine
 - JVM ensures uniform semantics across operating systems
 - Code is guaranteed to be portable